

Binary Tree

****You do not need to write the BTreeNode class, preorder()/postorder()/inorder()/tree_construction() etc. functions unless specified in the problem.****

****Just design the function specified in the problem.****

****No need to write driver code.****

****You may create helper functions to solve tasks****

****This document is a compilation of previous semester's exam (midterm/final) problems and lab quiz problems. Try to practice relevant problems from online judges as well.****

1.

Tasks from Lab 6 - [W](#) Lab 6 - Binary Tree.docx

Tasks from Book - [PDF](#) Practice Sheet 9 - Tree.pdf

2.

Write a **recursive** function **swap_child()** that takes the root of a binary tree, node's level and a number M as a parameter. The function will swap the left and right children of all the nodes at level M and above. Here, $0 < M < \text{height of the tree (root's height)}$. Consider, the Node class for Binary Tree already defined with elem, left and right variables. **YOU CANNOT USE LIST OR DICTIONARY, any built-in function, global variables.**

Python Notation:

```
def swap_child(root, level, M):  
    # To do
```

Function Call :

swap_child(root, 0, 2). Here root refers to the tree below.

Input Tree	Resulting Tree	Explanation
<pre> A / \ B C / \ / \ D E F / \ / \ G H I J</pre>	<pre> A / \ C B / \ / \ F E D / \ / \ J I G H</pre>	Here $M = 2$ and all the nodes from level 2 and above are swapped left with right. Here above means the level that situated at a higher position of the tree

3.

Given the **array representation** of a binary tree: [None value means the node is empty]

[None, 19, 22, 10, 32, 20, None, 30, None, 5, None, 7, None, None, 2, -2]

- Draw** the binary tree. [2 marks]
- Write** the post-order and pre-order traversal sequence of the tree. [1 marks]
- Use the **pre-order** traversal sequence in part b to **insert** the elements in that order in an initially empty binary search tree, and show the resulting binary search tree. Note: Consider the first element of the pre-order sequence as the root. [2 marks]
- Perform the following operations **step by step sequentially** on the Binary Search Tree you created in part c. [2 marks]
 - Delete node 5 with the help of its successor.
 - Delete node 20 with the help of its predecessor.

4.

Given a Binary Tree, complete a function that finds if the given Binary Tree is a SumTree or not. A SumTree is a Binary Tree where the value of a node is equal to the sum of the nodes present in its left subtree and right subtree. An empty tree is SumTree and the sum of an empty tree can be considered as 0. A leaf node is also considered as SumTree. The function should return True if it is a SumTree and otherwise False.

Consider the Node class for Binary Tree already defined with elem, left and right variables. You can use helper functions.

YOU CANNOT USE LIST OR DICTIONARY, any built-in function, or global variables.

Python Notation:

```
def sumTree(root):  
    // To do
```

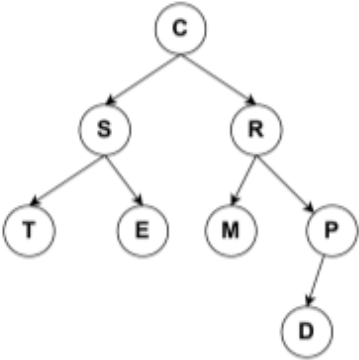
Here root refers to the tree below.

Sample Input	Sample Output	Explanation
<pre> 26 / \ 10 3 / \< \ 4 6 3</pre>	True	For 26 Sum of left(20)+sum of right(6)=26 For 10 Sum of left(4)+Sum of right(6)=10 For 3 Sum of left(0)+Sum of right(3)=3

5.

You are given a binary tree where each node stores a character. You are given the root of a binary tree and a key as inputs. The key is valid if there is a path from the root to a leaf node in the given tree. Now, your Task is to implement a **recursive function** named **is_Valid(root, key)** that determines if the key is valid or not. The function should return True if the key is valid and False otherwise.

Consider the Node class for Binary Tree already defined with elem, left and right variables. You can use helper functions and extra parameters if required. You can not use any other data structure than the binary tree.

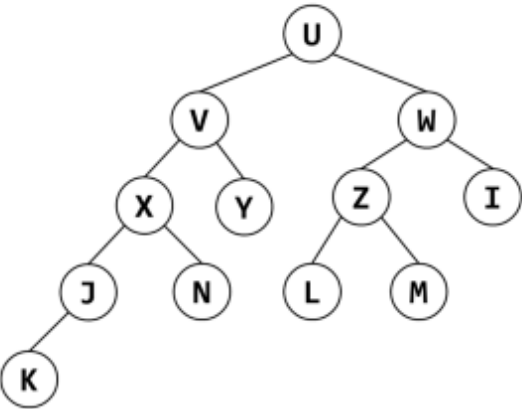
Sample Input	Sample Output	Explanation
	Key: CSE Output: True Key: CRA Output: False Key: CRP Output: False	For CSE there is a path from root to a leaf node where adding the characters of the nodes makes CSE. So the key is Valid. For CRA & CRP, there is no such path from root to a leaf node that creates the key. So they are invalid. For CRP the Node P is not leaf node hence it is ignored.

Python Notation	Java Notation
<pre>def is_Valid(root, key): # To Do</pre>	<pre>public boolean is_Valid(Node root, String key) { // To Do }</pre>

6.

You are given a binary tree where each node stores a character. You need to generate a key based on the leaf nodes positioned at the even levels **from right to left**. Your task is to implement a **function/method** named **build_key** that takes the root of the binary tree and constructs the key string based on the criteria above and finally returns it. If there are no leaf nodes on even levels, then the function/method should return an empty string.

Consider the Node class for Binary Tree already defined with elem (Type: String), left (Type: Node), and right (Type: Node) variables. **You can use helper functions or extra parameters in the given function if required. You can not use any other data structures than the given binary tree.**

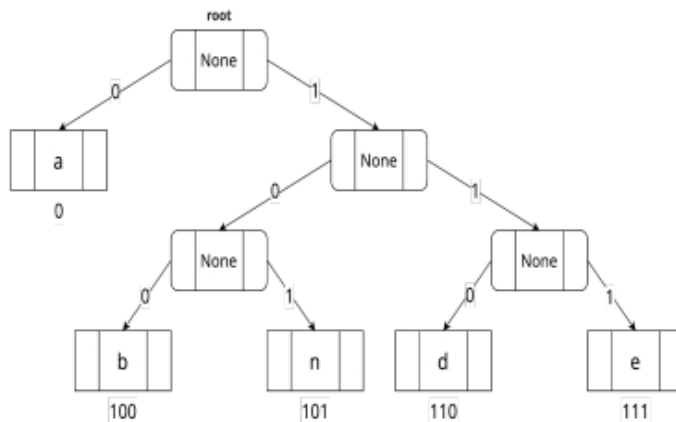
Input tree	Returned String	Explanation
	IYK	All the leaf nodes from right to left are: I, M, L, Y, N, K However, only I, Y, K are on even levels. Hence the output string is "IYK"

Python Notation	Java Notation
<pre>def build_key(root): #To do</pre>	<pre>static String build_key(Node root){ //To do }</pre>

7.

Question – 6: Bonus [5 Points]

The binary tree, called the Huffman coding tree, has all the possible valid characters as the leaves and all the internal nodes contain None/null. **Each transition to a left child represents a string 0-value bit and each transition to a right child from the parent represents a string 1-value bit. The bit representation of a character is the sequence of 0s and 1s you get through the path from the root to the leaf holding the characters.**



Here, the representation of **a** would be 0. Since we're moving only to the left 1 time. Similarly, **n** would be 101 since we're moving right, left, and right, and **d** would be 110 since we're moving right, right, left.

Each node of the Huffman encoding tree has four properties, a parent property to hold a reference of the parent of the current node, a left and a right to refer to the two children, and finally, a value property which is a character for the leaves and None/Null for the internal nodes.

Your task is to implement the following recursive function that takes the root of a Huffman encoding tree and a string text as parameters and returns a string representing the compressed bit encoding of the input string.

[The parent property of the node is absolutely not necessary to solve the task. But you can optimize the solution of this by using the node's parent property]

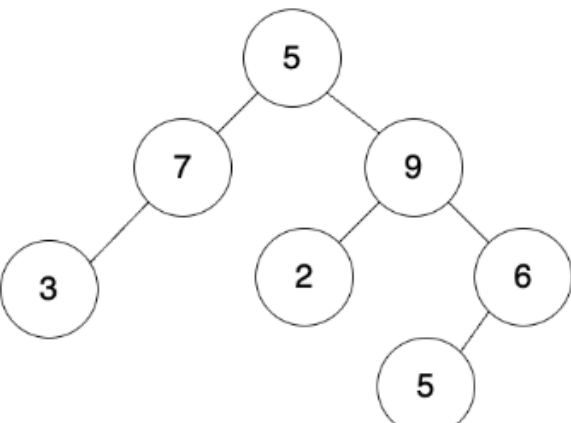
Note: You can create other recursive helper functions and call them inside the given input function. You can also use loops in your recursive functions. **There is no restriction on using other data structures in this problem.**

Sample Function Call	Sample Returned String	Explanation
<code>getEncoding(root, "and")</code>	<code>"0101110"</code>	Here, the bit representation of strings of "a", "n", "d" are "0", "101", "110" respectively. So, they're concatenated and shown in the output.

Python Notation	Java Notation
<pre>def getEncoding(root, st): #To do</pre>	<pre>static String getEncoding(Node root, String st){ //To do }</pre>

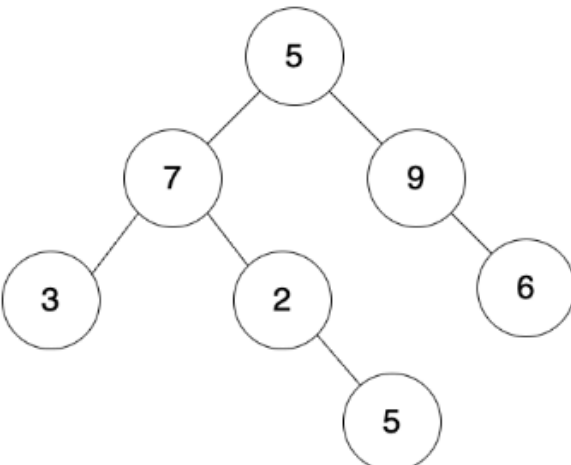
8.

You are an astronaut navigating through dense asteroid fields on your journey to the Andromeda galaxy. Your map represents a binary tree. Mission control said to find the safe path, the sum of energy signatures denoted by the node values. Your task is to write a **recursive** function **left_energy_signature(root)** that takes the root of a binary tree and returns the sum of all nodes that have **only a left child** and that left child is a **leaf node**.

Sample Input	Sample Output
	Node(5): Two childs, rejected Node(7): Only left child, left child is leaf, Accepted. Node(3): No left child, rejected Node(9): two childs, rejected Node(6): Only left child, left child is leaf, Accepted. Returns, Sum = 7 + 6 = 13

9.

You are an astronaut navigating through dense asteroid fields on your journey to the Triangulum Galaxy. Your map represents a binary tree. Mission control said to find the safe path, the sum of energy signatures denoted by the node values. Your task is to write a **recursive** function **right_energy_signature(root)** that takes the root of a binary tree and returns the sum of all nodes that have **only a right child** and that right child is a **leaf node**.

Sample Input	Sample Output
	Node(5): Two childs, rejected Node(9): Only right child, right child is leaf, Accepted. Node(6): No right child, rejected Node(7): Two childs, rejected Node(2): Only right child, right child is leaf, Accepted. Returns, Sum = 9 + 2 = 11

10.

Question: You are given the roots of two binary trees. Write a recursive function `is_identical(tree1_root, tree2_root)` that finds whether these trees are identical or not. Return true if the trees have the same structure and the same values at corresponding nodes, otherwise false.

Note: The function should not use any loops. You may use any number of helper functions as needed. Assume the `TreeNode` class is provided, where each node contains an integer value in the `element` attribute and pointers to the left and right child nodes.

Sample Input (only roots of trees will be given)	Sample Output
<pre> 1 1 / \ / \ 2 3 2 3 / \ / \ 4 5 4 5</pre>	true
<pre> 1 1 / \ / \ 2 3 3 2 / \ \ / 4 5 5 4</pre>	false

11.

Question: You are given the root of a binary tree. Write a recursive function `is_symmetric(root)` that checks whether the tree is symmetric around its centre. Return True if the tree is symmetric, otherwise return False. A binary tree is symmetric if the left subtree is a mirror reflection of the right subtree.

Note: The function should not use any loops. You may use any number of helper functions as needed. Assume the `TreeNode` class is provided, where each node contains an integer value in the `element` attribute and pointers to the left and right child nodes.

Sample Input (only the root of the tree will be given)	Sample Output
<pre> 1 / \ 2 2 / \ / \ 3 4 4 3</pre>	True
<pre> 1 / \ 2 2 \ \ 3 3</pre>	False

12.

You are given two binary trees, tree1 and tree2, with the same structure (same shape, possibly different node values). You are also given an integer divisor k.

Your task is to write a recursive function that traverses both trees simultaneously and updates the first tree (tree1) according to the following rules:

If the values at the same node position are equal:

- If divisible by k, replace the value in tree1 with 100.
- If not divisible, keep it unchanged.

If the values are not equal:

- Replace the value in tree1 with the sum of both node values.

Assume that the Node class, having data, left, and right attributes, is already given.

Sample Input	Sample Output
K = 3 Tree1: 6 / 5 9 / 2 3 Tree2: 6 / 1 4 / 2 7	100 / 6 13 / 2 10 ← 6 == 6 and 6 % 3 == 0 → 100 ← 5 + 1 = 6 and 9 + 4 = 13 ← 2 == 2 and 2 % 3 != 0 → unchanged ; 3 + 7 = 10

13.

Question:

You are given two binary trees, tree1 and tree2, with the same structure (same shape, possibly different node values). You are also given an integer divisor k.

Your task is to write a recursive function that traverses both trees simultaneously and updates the second tree (tree2) according to the following rules:

If the values at the same node position are not equal:

- If the tree2 node value is divisible by k, replace the value in tree2 with 100.
- If not divisible, keep it unchanged.

If the values are equal:

- Replace the value in tree2 with the sum of both node values.

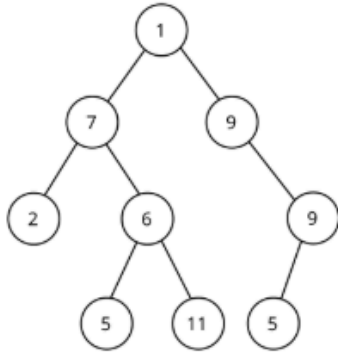
Assume that the Node class, having data, left, and right attributes, is already given

Sample Input	Sample Output
K = 3 Tree1: 6 / 5 9 / 2 3 Tree2: 6 / 1 6 / 2 9	12 / 1 100 / 4 100 ← 6 == 6 → 6 + 6 = 12 ← 6 != 9 and 6 % 3 == 0 → 100 ← 2 == 2 → 2 + 2 = 4 and 9 != 3 and 9 % 3 == 0 → 100

14.

Write a recursive function `max_path_sum()` that takes the root of the binary tree where each node contains a positive integer value. The function should traverse the tree from the root node to all leaf nodes and return the maximum path sum. *Note: A path sum is defined as the total sum of node values from the root to a leaf. A leaf node is a node that has no left or right child.*

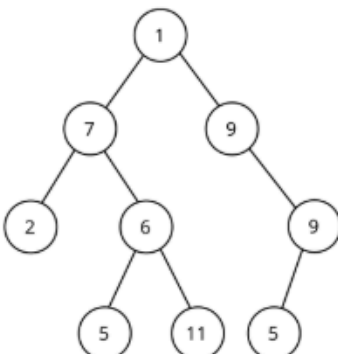
[You are not allowed to use any global variables or built-in functions. You can use multiple parameters and helper functions]

Input Tree	Output	Explanation
	25	Sum of node values for each root-to-leaf path and find the maximum path sum. Paths and their Sums: • $1 \rightarrow 7 \rightarrow 2 = 1 + 7 + 2 = 10$ • $1 \rightarrow 7 \rightarrow 6 \rightarrow 5 = 1 + 7 + 6 + 5 = 19$ • $1 \rightarrow 7 \rightarrow 6 \rightarrow 11 = 1 + 7 + 6 + 11 = 25$ • $1 \rightarrow 9 \rightarrow 9 \rightarrow 5 = 1 + 9 + 9 + 5 = 24$ The maximum path sum is 25, which comes from the path $1 \rightarrow 7 \rightarrow 6 \rightarrow 11$.

15.

Write a recursive function `min_path_sum()` that takes the root of the binary tree where each node contains a positive integer value. The program should traverse the tree from the root node to all leaf nodes and return the minimum path sum. *Note: A path sum is defined as the total sum of node values from the root to a leaf. A leaf node is a node that has no left or right child.*

[You are not allowed to use any global variables or built-in functions. You can use multiple parameters and helper functions]

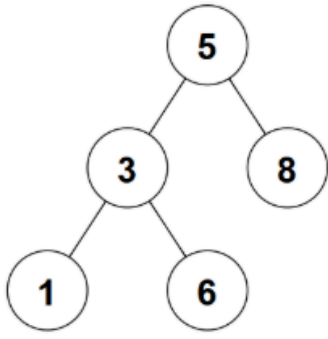
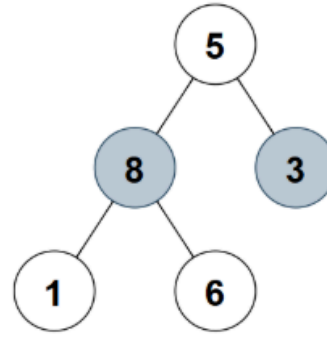
Input Tree	Output	Explanation
	10	Sum of node values for each root-to-leaf path and find the maximum path sum. Paths and Their Sums: • $1 \rightarrow 7 \rightarrow 2 = 1 + 7 + 2 = 10$ • $1 \rightarrow 7 \rightarrow 6 \rightarrow 5 = 1 + 7 + 6 + 5 = 19$ • $1 \rightarrow 7 \rightarrow 6 \rightarrow 11 = 1 + 7 + 6 + 11 = 25$ • $1 \rightarrow 9 \rightarrow 9 \rightarrow 5 = 1 + 9 + 9 + 5 = 24$ The minimum path sum is 10, which comes from the path $1 \rightarrow 7 \rightarrow 2$.

16.

Question:

You're working on an AI Game that processes Binary Trees. The tree has become unbalanced because some left children have become heavier than the right children on Odd levels. Your task is to write a *recursive function/method* **OddSwap(root)** that rebalances the tree by swapping sibling nodes at odd levels whenever the right child is "heavier" (has a greater value) than the left child. The **input** for the function is the **tree's root** only, so you can use helper functions.

YOU CANNOT USE LIST OR DICTIONARY. You cannot use any built-in function.

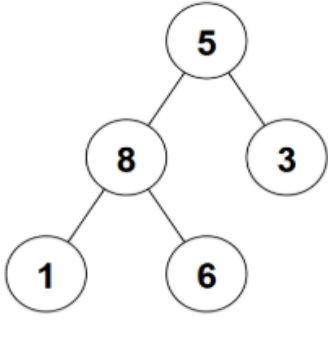
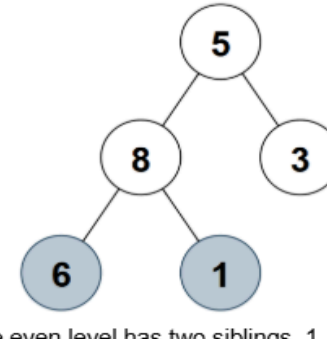
Sample Input	Sample Output
	 Here, the odd level has two siblings 3, 8, as $3 < 8$; the Nodes have been swapped.

17.

Question:

You're working on a Cut the Rope game that uses Binary Trees. The tree has become unbalanced because some right children have become heavier than the left ones on Even levels. Your task is to write a *recursive function/method*, **EvenSwap(root)**, that rebalances the tree by swapping sibling nodes at even levels whenever the right child is "heavier" (has a greater value) than the left child. The **input** for the function is the **tree's root** only, so you can use helper functions.

YOU CANNOT USE LIST OR DICTIONARY. You cannot use any built-in function.

Sample Input	Sample Output
	 Here, the even level has two siblings, 1, 6, as $1 < 6$; the Nodes have been swapped.

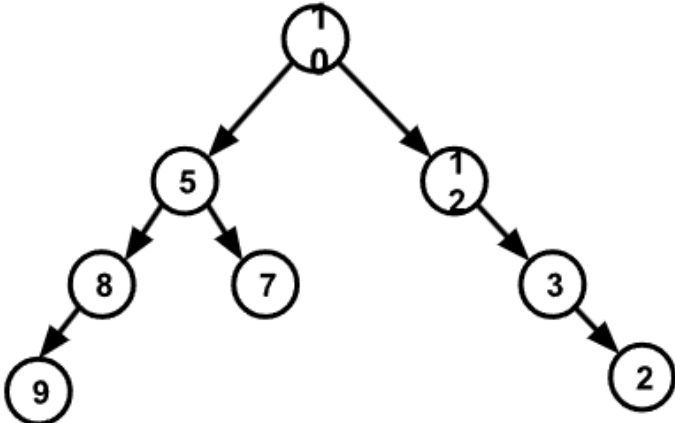
18.

Question: Alice and Bob are best friends who enjoy solving puzzles on binary trees. They came up with this game:

- Alice looks only at the left subtree of the root and sums up all the odd numbers he can find.
- Bob looks only at the right subtree of the root and sums up all the even numbers he can find.

As you already learned about the Binary tree, help them to calculate the absolute difference between the sum of Alice's odd numbers and Bob's even numbers. Write a recursive function/method **subtree_difference()** that takes the root of the binary tree as a parameter and returns the absolute difference. Consider the Node class for Binary Tree already defined with `elem`, `left` and `right` variables. You can use helper functions.

YOU CANNOT USE LIST OR DICTIONARY. You cannot use any built-in function.

Sample Input	Sample Output
	7 Explanation: Summation of all odd number in left subtree = (5+7+9) =21 Summation of all even number in right subtree = (12+2) =14 absolute difference = 21-14 = 7

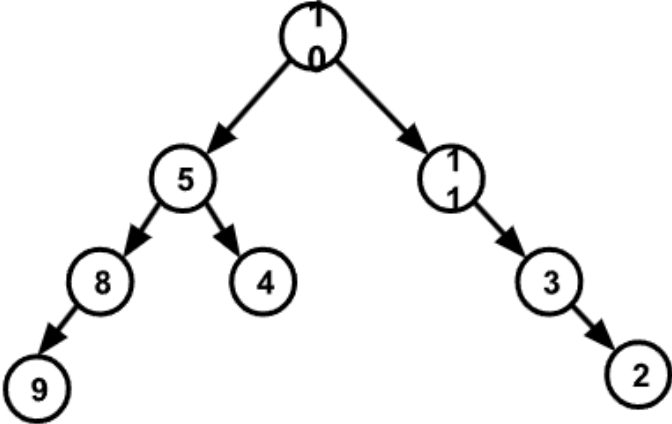
19.

Question: Alice and Bob are best friends who enjoy solving puzzles on binary trees. They came up with this game:

- Alice looks only at the left subtree of the root and multiplies all the even numbers he can find.
- Bob looks only at the right subtree of the root and multiplies all the odd numbers he can find.

As you already learned about the Binary tree, help them to calculate the absolute difference between the product of Alice's odd numbers and Bob's even numbers. Write a recursive function/method **subtree_difference()** that takes the root of the binary tree as a parameter and returns the absolute difference. Consider the Node class for Binary Tree already defined with elem, left and right variables. You can use helper functions.

YOU CANNOT USE LIST OR DICTIONARY. You cannot use any built-in function.

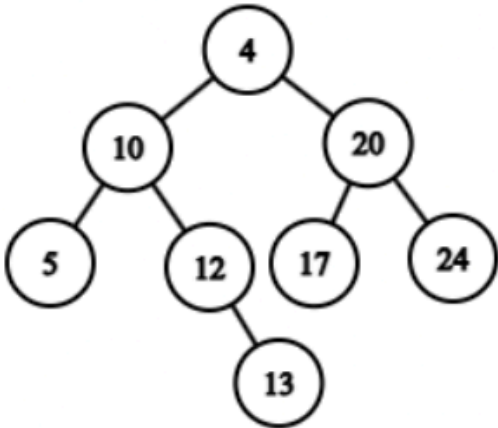
Sample Input	Sample Output
	1 Explanation: product of all even number in left subtree = $(8*4) = 32$ product of all odd number in right subtree = $(11*3) = 33$ absolute difference = $ 32-33 = 1$

20.

You are given a binary tree. Your task is to find the **size of the largest subtree that is a Binary Search Tree (BST)**.

YOU CANNOT USE LIST OR DICTIONARY. You cannot use any built-in function except for max function. You can create helper functions if needed.

[Hint: You may need three functions, one for checking if the tree is a BST and one for calculating the size of the tree and one for returning the largest BST]

Sample Input	Sample Output
 <pre>graph TD; 4((4)) --- 10((10)); 4 --- 20((20)); 10 --- 5((5)); 10 --- 12((12)); 12 --- 13((13)); 20 --- 17((17)); 20 --- 24((24));</pre>	4 Explanation: The whole tree is not a BST Left subtree is a BST with size 4 Right subtree is a BST with size 3 Hence, the largest subtree that is a BST is of size 4

21.

1. Write a function **symmetricFactorDiff(root, x)** that takes the root of a BST and a node x.

- For the subtree rooted at x, consider all nodes with a symmetric sibling.
- For each symmetric pair, if one node is a factor of the other, add the absolute difference to the total sum.
- Ignore nodes without symmetric siblings.

Note: You must use recursion. You may use any number of helper functions as needed. You may assume that the `TreeNode` class has been defined for you.

Input	Output
<pre> 8 / \ 6 12 / \ \ 3 7 24 x = 8</pre>	6 + 21 = 27 Explanation: 6 ↔ 12 12 % 6 == 0 absolute difference: 6 3 ↔ 24 24 % 3 == 0 absolute difference: 21 7 no symmetric sibling ignored

Short Question [5 marks]

2a. What is a balanced binary tree?

2b. What is the time complexity of searching a node in a balanced BST?

2c. What is the maximum number of nodes in a binary tree of height "H"?

2d. What does the in-order traversal always produce in a BST?

2e. What is the degree of a leaf node in a Binary tree?

22.

Create a function called `strangeFunc` which takes a Binary tree root and an integer `k` as input: `strangeFunc (Node root, int k)`. Then recursively, replace each node by the absolute difference of the child nodes if the node is divisible by `k`, else double them. If a node has a missing child, do not apply this algorithm to it or its subtrees.

(Assume node class is already implemented)

Absolute Function

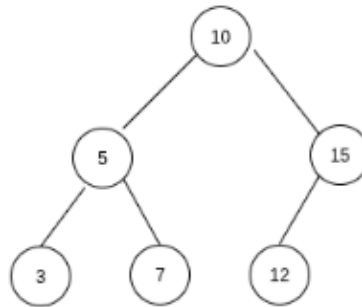
Python: `abs(num)`

Java: `Math.abs(num)`

Sample Input	Sample Output
<code>strangeFunc (root, 3)</code> <pre>graph TD; 2 --> 3; 2 --> 5; 3 --> 6; 3 --> 7; 6 --> 10; 5 --> 8; 5 --> 9; 8 --> 11; 8 --> 12; 9 --> 14;</pre>	Here 2 is not divisible by 3, hence it is doubled. 3 is divisible by 3, hence it is replaced by the absolute difference of its child. 6 and 9 both have missing children, so the algorithm is not applied to those subtrees.. <pre>graph TD; 4 --> 1; 4 --> 10; 1 --> 6; 1 --> 7; 6 --> 10; 10 --> 16; 10 --> 9; 16 --> 11; 16 --> 12; 9 --> 14;</pre>

23.

- 1) If there are a total of 9 Nodes in a tree, then what will be the total Edges in that tree?



- 2) If we mirror the above tree, what will be the new post order traversal?

- 3) What is the Degree of the above original tree?

- 4)

FUNCTION height(root)
IF root IS NULL THEN
RETURN 0
RETURN MAX(height(root.left), height(root.right))

Is this code correct for finding the Height of a Binary Tree? If not, what is wrong here?

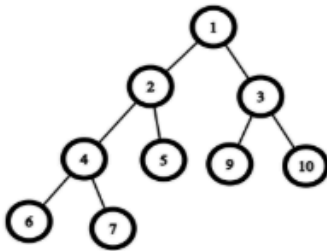
- 5) Construct the tree from the given array = [None, 'A', 'B', 'C', None, 'D', None, None]

24.

The following pseudocode is intended to count the total number of even nodes in a binary tree. Identify the errors and write the corrected lines only.

ERRORED CODE	CORRECTED LINES
<pre>FUNCTION evenNodes(node) IF node == NULL RETURN -1 leftEven = evenNodes(node.left) rightEven = evenNodes(node.right) IF node.data % 2 == 0 RETURN leftEven + rightEven ELSE RETURN leftEven + rightEven END FUNCTION</pre>	

Is the following binary tree a perfect binary tree? Explain.



ANSWER:

Q3. Suppose you are given the array representation of a binary tree: {null, 4, 9, 6, 16, null, 8, 20}. Now, draw the corresponding binary tree.

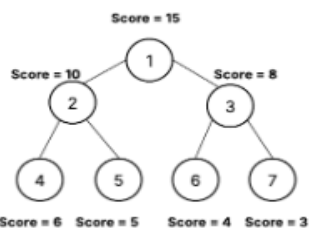
ANSWER:

25.

The BRAC University Computer Club (BUCC) is organized as a binary tree, where each member has a unique ID and a contribution score representing their effort for the club. A member may have up to two direct juniors: the first junior is the left child, and the second junior is the right child.

Given the ID of a particular member, compute the total contribution score of that member and all of their direct and indirect juniors in the binary tree. If the given ID does not exist in the binary tree, return 0. Write a function `totalContribution()` that takes the root of the binary tree and the target ID as its parameters.

Note: You may use extra parameters or helper functions as much as you want. You are required to use recursion. Consider the `BTNode` class is already implemented with the instance variables `left (BTNode)`, `right (BTNode)`, `score (int)`, `ID (int)`.

Sample Tree	Sample Function Call	Sample Output
	(Python) <code>print(total_contribution(root, 1))</code>	51
	(Java) <code>System.out.println(totalContribution(root, 1));</code>	Explanation: (15+10+8+6+5+4+3)=51
	(Python) <code>print(total_contribution(root, 2))</code>	21
	(Java) <code>System.out.println(totalContribution(root, 2));</code>	Explanation: (10+6+5)=21

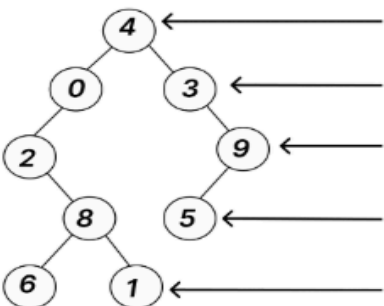
26.

You're developing an AI agent that observes a **Binary Tree battlefield** from the right side. The agent can only "see" nodes that are visible from the right side of the tree.

Your task is to write a *recursive function/method* `RightView(root, level, max_Level)` that prints a list of node values the agent can see, ordered from top to bottom.

- Here, root: The current node, level: The current level of the node in the tree, max_level: A variable to store the deepest level encountered so far.
- Initially, root = the root of the binary tree, level = 0, and max_level = -1.
- You may use helper functions if needed.
- Think of the tree levels as "rows" and always prioritize seeing the rightmost node first at each row.

YOU CANNOT USE LIST OR DICTIONARY. You cannot use any built-in function.

Sample Input	Sample Output
	4 3 9 5 1 Explanation: As the arrows indicate, if you see the graph from the right side, then the first nodes that you encounter at each level are the nodes you need to print from top to bottom.

27.

Question 3: CO4 [8 Points]

Complete the function below, which will receive the root of a binary tree as a parameter and return the second-highest value from the tree. Assume there will always be a second-highest value in the tree. **The node class has an elem (Integer type), left (Node type), and right (Node type) variable.**

Page 2 of 4

Set-A

Restrictions:

- You cannot use any global variables and cannot create any other data structures (e.g, Array length more than 2, linked list, etc).
- You cannot change the number of parameters of the second_max function.
- You can design and use helper functions. You can use an array of length 2 if you want to.

Input tree	Returned Value	Explanation
<pre> 30 / \ 50 10 / 40</pre>	40	In the given binary tree, the maximum value is 50 and the second-highest value is 40.

Python Notation	Java Notation
<pre>def second_max(root): #To do</pre>	<pre>static int second_max(Node root){ //To do }</pre>

28.

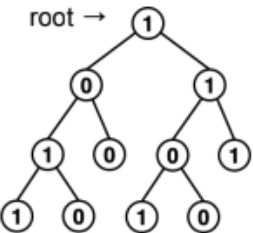
Question 3: CO4 [8 Points]

In a maze game, a player starts at the entry point of the maze, which is the root of a binary tree. Each node contains either a 0 or a 1. Each path from the root down to a leaf represents one possible way out through the maze. The player is given a sequence of 0s and 1s. They must check whether following that sequence will allow them to escape the maze by reaching a leaf node.

Write a function `findPath()` to help the player. This function will take the **root** of the binary tree, and the **sequence in an (int) array** to follow. The function will **return a string** "ESCAPED" if the sequence matches a complete path from the root to a leaf, "TRAPPED" if the sequence matches part of a path but ends at a non-leaf node, "LOST" if the sequence cannot be matched to any path in the tree.

Assume the Node class of the binary tree is already defined with the properties: (int) elem, (Node) left, (Node) right.

- You can take only one extra parameter or a helper function.
- You must solve this using recursion only.
- No extra data structures (array/linked list) or static/global variables are allowed.

Tree	Input	Returned String	Explanation
	seq = {1,0,1,1} findPath(root, seq)	ESCAPED	Matches path 1→0→1→1 Completely from root to leaf
	seq = {1,1,0} findPath(root, seq)	TRAPPED	Matches part of the path but ends at a non-leaf
	seq = {1,0,0,1} findPath(root, seq)	LOST	No such path in this tree

Python Notation	Java Notation
<pre>def findPath(root, seq): #To do</pre>	<pre>public static String findPath(Node root, int[] seq){ //To do }</pre>